

TI-4601 Bases de datos avanzados

Proyecto 1

Andrés Alfaro Mayorga - 2023396028

Andrés Uriza Lazo - 2021466844

Gabriel Guzmán Rojas - 2022090213

23 de septiembre de 2026

Motor: CockroachDB x3

Dominio: Servicio de banco, billetera regional.

1. Requisitos y regiones

Levantar este proyecto requiere:

- Docker Compose v5.5.0
- Docker Engine Community 29.8.0
- Puertos 26257 y 8080 libres
- 4 core CPU
- 4GB RAM
- 5GB almacenamiento de disco libre

Se cuenta con las siguientes regiones:

- cr-sj
- cr-limon
- us-east

2. Diseño de fragmentación y asignación

2.1. Esquema lógico

Cuadro 1: Cliente

id	Nombre	Documento	region_cliente
11111111-1111-1111-1111-111111111111	Harry	184295827	cr-sj
22222222-2222-2222-2222-222222222222	Ana	105820394	cr-sj
33333333-3333-3333-3333-333333333333	Michael	148295327	cr-limon
44444444-4444-4444-4444-444444444444	Laura	209384756	us-east

PK: id

Cuadro 2: Cuenta

id	Monto	Tipo	cliente_id	region_cliente
aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa	100000	corriente	11111111-1111-1111-1111-111111111111	cr-sj
bbbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb	20000	corriente	22222222-2222-2222-2222-222222222222	cr-sj
cccccccc-cccc-cccc-cccc-cccccccccccc	0	corriente	33333333-3333-3333-3333-333333333333	cr-limon
dddddddd-dddd-dddd-dddd-dddddddddddd	50000	corriente	44444444-4444-4444-4444-444444444444	us-east

PK: id

FK: cliente_id $\rightarrow$ Cliente(id)

Cuadro 3: Movimiento

id	cuenta_origen	cuenta_destino	monto	fecha	region_movimiento
(auto)	aaaaaaaa- aaaa-aaaa- aaaa- aaaaaaaaaaaa	bbbbbbbbb- bbbb-bbbb- bbbb- bbbbbbbbb	5000	2026-03-06	cr-sj
(auto)	aaaaaaaa- aaaa-aaaa- aaaa- aaaaaaaaaaaa	cccccccc-cccc- cccc-cccc- cccccccccccc	50000	2026-03-06	cr-sj

PK: id

FK: cuenta_origen $\rightarrow$ Cuenta(id)

FK: cuenta_destino $\rightarrow$ Cuenta(id)

Cuadro 4: ControlDisponibilidad

id	version	actualizado_en
1	0	2026-09-09

PK: id

2.2. Fragmentaciones

2.2.1. Cliente

$\text{Cliente1} = \sigma_{\text{REGION_CLIENTE} = \text{'cr-sj'}}(\text{Cliente})$

$\text{Cliente2} = \sigma_{\text{REGION_CLIENTE} = \text{'cr-limon'}}(\text{Cliente})$

$\text{Cliente3} = \sigma_{\text{REGION_CLIENTE} = \text{'us-east'}}(\text{Cliente})$

2.2.2. Cuenta

$\text{Cuenta1} = \sigma_{\text{REGION_CUENTA} = \text{'cr-sj'}}(\text{Cuenta})$

$\text{Cuenta2} = \sigma_{\text{REGION_CUENTA} = \text{'cr-limon'}}(\text{Cuenta})$

$\text{Cuenta3} = \sigma_{\text{REGION_CUENTA} = \text{'us-east'}}(\text{Cuenta})$

2.2.3. Movimiento

$\text{Movimiento1} = \sigma_{\text{REGION_MOVIMIENTO} = \text{'cr-sj'}}(\text{Movimiento})$

$\text{Movimiento2} = \sigma_{\text{REGION_MOVIMIENTO} = \text{'cr-limon'}}(\text{Movimiento})$

$\text{Movimiento3} = \sigma_{\text{REGION_MOVIMIENTO} = \text{'us-east'}}(\text{Movimiento})$

2.3. Tabla de verificación

Cuadro 5: Verificación

Propiedad	Se cumple?	Justificación
Compleitud	Si	Se cumple ya que no se permiten valores nulos para PII de Cliente, Cuenta debe tener un cliente y movimiento debe pertenecer a dos cuentas, la origen y destino. Cada fragmentación corresponde a las 3 regiones que existen.
Reconstrucción	Si	Si hacemos unión de la región de los fragmentos, se puede reconstruir la db para cada una de las tres tablas.
Disyunción	Si	Se fragmentó horizontalmente cada una de las tablas en sus regiones, es decir que se excluyen mutuamente, al no dejar ninguna de lado.

2.4. Réplica

Se replica unicamente la tabla **ControlDisponibilidad**, con el factor de replicacion por defecto $RF = 3$, esto con el fin de poder medir limpiamente el RTO y RPO.

Los criterios de costos y diseño seleccionados consideran los siguientes aspectos:

1. **Ubicación del Leaseholder:** Se procura mantener el leaseholder físicamente en la región donde se registra la mayor concentración de consultas para un fragmento determinado.
2. **Factor de Réplica ($RF = 3$):** Se utiliza para garantizar *alta disponibilidad y tolerancia a fallos* frente a la caída de nodos, manteniendo una copia exacta en cada una de las regiones (cr-sj, cr-limon y us-east).
3. **Optimización de Rendimiento y Costos:**
 - **Reducción de *joins* remotos:** Gracias a la fragmentación horizontal por región, los datos relacionados se encuentran accesibles de forma local, mejorando rendimiento.
 - **Eficiencia en lecturas:** Se aumenta la eficiencia al repartir las consultas mediante *follower reads* y *stale reads*, aliviando el tráfico del nodo principal.
 - **Compromisos del sistema:** No obstante, se reconoce que la arquitectura distribuida no elimina por completo las lecturas ni las escrituras remotas, ya que las transacciones globales o aquellas que cruzan fronteras de región requerirán coordinación de red.

2.5. Descripción de la asignación

La asignación se realiza por región, para las tablas Cliente, Cuenta y Movimiento, para cada una de las tres regiones, se asignan los fragmentos a cada site `cr-sj`, `cr-limon`, `us-east`.

Al utilizar además REGIONAL BY ROW se asegura que la PII de la tabla Cliente existe solo en su región de apertura, al definirse la región del cliente en su tupla.

3. Implementación

3.1. Definición de operación local y cruza región

Una operación local es aquella en la que la transacción interactúa únicamente con datos ubicados en la misma región del gateway, mientras que una operación de cruza región (remota) requiere coordinar con datos de una región distinta.

3.2. Perfil de Recursos y Entorno

Las pruebas se ejecutaron bajo un perfil de laboratorio reducido (entorno de desarrollo local), donde los tres nodos del clúster de CockroachDB y la aplicación cliente corren de manera concurrente en la misma máquina física (Asus Vivobook) mediante contenedores de Docker compartiendo los mismos límites de recursos de CPU y memoria.

3.3. Descarte de Cold Start

Para evitar sesgos iniciales debidos al establecimiento de conexiones, compilación de planes de ejecución o carga inicial de caché en el motor (cold start), se ejecutaron 5 rondas de warm-up previas, cuyos tiempos fueron descartados antes de iniciar el registro formal de las muestras.

3.4. Resultados

Los resultados obtenidos sobre $n = 50$ corridas válidas se resumen en la Tabla 6

Cuadro 6: Rendimiento de operaciones locales y remotas (p50 y p99).

Operación	Localidad	Región Home	n	p50 (ms)	p99 (ms)
Lectura (Read)	Local	cr-sj	50	0.657	0.959
Lectura (Read)	Remota	cr-sj $\rightarrow$ cr-limon	50	1.366	1.750
Escritura (Write)	Local	cr-sj	50	3.627	7.176
Escritura (Write)	Remota	cr-sj $\rightarrow$ cr-limon	50	5.384	8.824

3.5. Inyección de Latencia

No se aplicó inyección artificial de latencia de red en el entorno de laboratorio reducido, por lo que los valores de tiempo obtenidos corresponden exclusivamente a la sobrecarga inherente de red y consenso local de Docker.

4. Falla de Sitio (Nodo)

4.1. Precondición

El experimento se realiza sobre `ControlDisponibilidad`, la única tabla del esquema que no utiliza `REGIONAL BY ROW`. Esto es intencional: en un clúster con un solo nodo por región, una tabla `REGIONAL BY ROW` no garantiza que sus 3 réplicas queden repartidas una por nodo (el rango podría quedar subreplicado). Al dejar `ControlDisponibilidad` sin fragmentar, Cockroach le asigna el factor de replicación por defecto del clúster ($RF = 3$), y antes de iniciar la falla se verificó con `SHOW RANGES` que `voting_replicas` contenía efectivamente los 3 nodos, con el *leaseholder* en `crdb-1` (región `cr-sj`).

4.2. Protocolo

1. Se inició una sonda de escritura continua sobre `ControlDisponibilidad(id=1)`, confirmando estado sano (*before-stop ok*).
2. Se detuvo el nodo `crdb-1` con `docker stop --timeout 0`, registrando el instante exacto de la falla.
3. Se cronometró hasta la primera escritura confirmada nuevamente.
4. Se calculó el RTO observado y se verificó el RPO comparando la versión de la fila antes y después del incidente.

4.3. Cronología observada

Cuadro 7: Cronología de la falla de `crdb-1`

Evento	Resultado
Escrituras antes de la falla	OK, 10–18 ms de latencia
<code>docker stop</code> en <code>crdb-1</code>	Instante de referencia (t_0)
Intento 1 post-falla	Error (<code>QueryCanceled</code> , timeout 2004 ms)
Intento 2 post-falla	Error (<code>QueryCanceled</code> , timeout 2008 ms)
Intento 3 post-falla	OK – primera escritura confirmada (310 ms)
Escrituras posteriores	OK, latencia normal (10–18 ms)

4.4. Cálculo de RTO

$$\text{RTO} = t_{\text{primer OK posterior}} - t_0 = 4187,3 \text{ ms}$$

Este tiempo corresponde al proceso interno de Raft: detección de que **crdb-1** dejó de responder, elección de un nuevo líder entre **crdb-2** y **crdb-3** (2 de 3 nodos, mayoría suficiente), y reparación de los *followers* antes de volver a aceptar escrituras.

4.5. Verificación de RPO

Tras restaurar el nodo, se consultó el estado final de la fila:

Cuadro 8: Estado de `ControlDisponibilidad` tras la recuperación

id	version	actualizado_en
1	224	2026-09-14 22:14:51.70

La columna **version** nunca retrocedió respecto al valor confirmado antes de la falla, y las escrituras posteriores a la recuperación continuaron incrementándola de forma monótona. Esto confirma $\text{RPO} \approx 0$ para operaciones confirmadas.

4.6. Interpretación

Los 2 errores `QueryCanceled` registrados entre la falla y la recuperación **no representan pérdida de datos**: son intentos que nunca llegaron a confirmarse, por lo que no violan la garantía de RPO. Raft solo promete que lo ya **comprometido** (confirmado por mayoría) sobrevive a la falla de una minoría de nodos – exactamente lo observado: con 2 de 3 nodos vivos, el sistema siguió aceptando y confirmando escrituras sin perder ningún dato previamente confirmado.

5. Crítica

5.1. Arquitectura Alternativa: 1 Nodo Primario + Réplica de Lectura

Como contraparte al clúster distribuido de tres nodos, la alternativa consiste en una base de datos centralizada compuesta por un único nodo primario (*master*) encargado de procesar todas las operaciones de escritura (*INSERT*, *UPDATE*, *DELETE*) y una réplica secundaria (generalmente asíncrona) destinada exclusivamente a atender las consultas de lectura (*SELECT*).

En este modelo, las transacciones locales se resuelven de manera directa y eficiente sin los retardos asociados al protocolo de consenso multi-región Raft. Las réplicas de lectura permiten escalar el rendimiento de las consultas y descongestionar el nodo principal; sin embargo, esta topología introduce limitaciones críticas en términos de arquitectura de alta disponibilidad:

- **Punto Único de Escritura:** Toda la carga transaccional y la responsabilidad de consistencia recaen de forma exclusiva en el nodo primario. Si este nodo sufre una caída física o un fallo de software, el sistema pierde por completo la capacidad de aceptar escrituras.
- **Inutilidad de las Réplicas para Escrituras:** Aunque el clúster mantenga una réplica secundaria activa, esta operan en modo de solo lectura. No puede asumir automáticamente el rol de escritura sin un proceso manual o un orquestador externo, lo cual interrumpe la disponibilidad del servicio.
- **Limitación en la Latencia Geográfica:** Al tener una sola réplica, esta no puede estar en varios lugares a la vez. Si se coloca junto al primario, los usuarios de otras regiones sufren de alta latencia. Si se pone en otra región para acercarla a los usuarios, de todos modos las escrituras deben viajar obligatoriamente hasta el nodo central, y las lecturas sufren un desfase de sincronización (*replication lag*).

5.2. Comparación Multidimensional (Latencia, Complejidad y Costos con Números)

Para saber si vale la pena distribuir la base de datos, comparamos el clúster de 3 nodos (CockroachDB) contra una arquitectura mucho más simple: un solo servidor de PostgreSQL con una réplica de lectura.

5.2.1. Rendimiento y Latencia

En el clúster distribuido, escribir tardó en promedio **3.627 ms** si la operación se hacía en el mismo nodo donde vive el dato, y **5.384 ms** si tenía que viajar a otra región. Esa diferencia es el costo de ponerse de acuerdo entre varios servidores (el protocolo de consenso, Raft) antes de confirmar cada escritura.

En la arquitectura primario + réplica no existe esa distinción de localidad: como solo hay dos servidores fijos, toda lectura se atiende siempre en la réplica y toda escritura siempre en el primario, sin importar qué cuenta se consulte. Por eso aquí se reporta una sola medición por operación en vez de separar por región. Las lecturas contra la réplica tardaron en mediana **0.486 ms** (p99 de **0.967 ms**), mientras que las escrituras contra el primario tardaron **1.595 ms** (p99 de **2.427 ms**).

Cuadro 9: Latencias (p50 y p99) en la arquitectura primario + réplica (PostgreSQL).

Operación	Nodo	N	p50 (ms)	p99 (ms)
read	réplica	50	0.486	0.967
write	primario	50	1.595	2.427

5.2.2. Concurrencia: costo escondido de tener un solo primario

Esta velocidad tiene una letra pequeña: como todas las escrituras caen sobre un único servidor primario, ese nodo también es el único responsable de resolver los conflictos entre transacciones

concurrentes. Para medirlo, se lanzaron 40 hilos debitando 1 de la misma cuenta al mismo tiempo, comparando dos niveles de aislamiento sobre esa misma arquitectura primario + réplica:

Cuadro 10: Concurrencia sobre la misma cuenta con 40 hilos (arquitectura primario + réplica).

Aislamiento	Commits OK	Abortos por serialización	Saldo final	Saldo esperado
READ COMMITTED	40	0	996.0	960.0
SERIALIZABLE	38	152	962.0	962.0

Bajo `READ COMMITTED`, las 40 escrituras se confirmaron sin un solo error, pero el saldo final (996) quedó por encima de lo esperado (960): de los 40 débitos de 1 que deberían haberse aplicado, solo 4 sobrevivieron. Los otros 36 se perdieron silenciosamente porque dos transacciones podían leer el mismo saldo antes de que la otra escribiera, y la segunda escritura simplemente sobrescribía a la primera sin avisar — el clásico *lost update*. Ningún mensaje de error delató el problema; el sistema “funcionó” y aun así perdió dinero.

Bajo `SERIALIZABLE`, el motor detectó esas mismas carreras y las abortó: hubo **152** abortos por serialización en total (contando reintentos). Con hasta 8 reintentos por hilo, 38 de los 40 lograron finalmente completarse, y el saldo final (962) coincidió exactamente con lo esperado (1000 – 38). Los otros 2 hilos agotaron sus reintentos sin lograr comprometerse; el precio de garantizar consistencia bajo contención es que algunas transacciones simplemente no logran pasar, incluso reintentando.

CockroachDB evita este dilema de raíz: al repartir las escrituras entre varios nodos mediante particionamiento por filas, reduce la probabilidad de que dos transacciones compitan exactamente por la misma fila al mismo tiempo, y cuando sí compiten, resuelve el conflicto con el mismo mecanismo de aislamiento serializable que trae por defecto.

5.2.3. Complejidad Operativa

Un solo servidor con una réplica es mucho más simple de mantener: no hay que coordinar relojes entre servidores, no hay votaciones para ponerse de acuerdo, y no hay que preocuparse por mover datos automáticamente si un nodo falla. El clúster distribuido, en cambio, obliga a decidir con cuidado en qué región vive cada fila de datos (con `REGIONAL BY ROW`), lo que hace su día a día más complicado de operar.

5.2.4. Costo de Recursos (Misma Máquina)

Corriendo ambas arquitecturas en la misma laptop, el clúster de 3 nodos consume más CPU y RAM, porque los tres servidores se están enviando señales de vida (“heartbeats”) todo el tiempo entre sí. La arquitectura primario + réplica, al ser solo dos servidores sin ese intercambio constante, es más liviana.

5.3. Conclusión

5.3.1. ¿Está justificado el sistema distribuido?

Si solo miramos velocidad pura y uso de recursos, no: un servidor primario responde una escritura en menos de dos milisegundos, mientras que el clúster distribuido paga un costo extra por ponerse de acuerdo entre nodos.

Pero la velocidad no es el único criterio, y el experimento de concurrencia lo muestra con números concretos: sin aislamiento estricto, la arquitectura centralizada puede perder actualizaciones silenciosamente (36 de 40 débitos desaparecidos); con aislamiento estricto, evita esa corrupción pero paga el costo en transacciones abortadas bajo contención. A eso se suma que, en un negocio financiero, los datos de un cliente muchas veces deben guardarse en su propio país o región por ley, y el sistema no puede depender de un solo servidor que, si falla, se lleva todo con él. Por eso, aunque sea más lento y más complejo de operar, la arquitectura distribuida sí se justifica: cambia unos milisegundos de latencia por integridad de los datos bajo concurrencia, cumplimiento normativo, y no tener un único punto de falla.

6. Apéndices

6.1. Roles del equipo

Andrés Alfaro: Encargado de hacer revisión de la documentación, estructura general y redacción, agregar parte de critica distribución. Análisis de resultados de latencia. Recolección de evidencias falla de sitio.

Andrés Uriza: Captura de evidencia resultados latencia de E2. Diseño de tablas y fragmentación.

Gabriel: Diseño de la fragmentación en `schema.sql` y de la tabla `ControlDisponibilidad` para E4. Desarrollo de `verify_cluster.py` y `falla_nodo.py`. Ejecución del experimento de falla de nodo y recolección de evidencia de E2 y E4.